KuzuDB and NetworkX integration for graph analytics

KuzuDB provides native NetworkX conversion through its `get_as_networkx()` method, enabling a seamless workflow from Cypher queries to Python graph analytics. The embedded graph database—now continued as Ryugraph following KuzuDB's October 2025 archival—offers **zero-copy DataFrame integration**, automatic disk spilling for large queries, and vectorized execution that benchmarks up to **188x faster than Neo4j** on path-finding queries.

Native NetworkX conversion eliminates manual data wrangling

The KuzuDB Python API provides direct graph conversion through the `QueryResult` class. After executing a Cypher query that returns nodes and relationships, calling `get_as_networkx(directed=True)` produces a NetworkX DiGraph (or Graph for undirected) with all node and edge properties automatically transferred as attributes.

```
import kuzu
import networkx as nx

db = kuzu.Database("./transaction_db")
conn = kuzu.Connection(db)

# Extract subgraph via Cypher
result = conn.execute("""
    MATCH (c:Client)-[t:TransactedWith]->(m:Merchant)
    WHERE t.is_disputed = true
    RETURN *;
""")

# Convert to NetworkX DiGraph
G = result.get_as_networkx(directed=True)

# Run any NetworkX algorithm
pagerank = nx.pagerank(G)
centrality = nx.betweenness_centrality(G)
```

Nodes in the resulting graph use composite identifiers combining the label and primary key (e.g., `'Client_35'`). All properties from KuzuDB transfer directly—node properties become `G.nodes[node_id]` attributes, edge properties become `G.edges[u, v]` attributes. The method handles heterogeneous graphs containing multiple node and relationship types, making it suitable for complex property graph schemas.

Cypher patterns optimized for subgraph extraction

The most effective Cypher patterns for NetworkX workflows return complete graph structures rather than scalar projections. Using `RETURN *` or explicitly returning node and relationship variables ensures the conversion receives the full graph topology.

Variable-length paths use Kleene star syntax with bounds:

```
-- 1 to 3 hops with intermediate filtering
MATCH p = (a:User)-[:Follows*1..3 (r, n | WHERE r.since < 2022 AND n.age >
30)]->(b:User)
WHERE a.name = "Alice"
RETURN *;
```

Multi-hop neighborhood extraction for ego networks:

```
MATCH (center:Person {id: $target_id})-[r*1..2]-(neighbor)
RETURN *
LIMIT 1000;
```

For **large subgraph exports**, pagination prevents memory issues:

```
MATCH (n:Person) RETURN n.* ORDER BY n.id SKIP 100000 LIMIT 100000;
```

DataFrame integration provides a powerful intermediate layer

KuzuDB's zero-copy integration with Polars, Pandas, and PyArrow enables sophisticated ETL pipelines. Results can flow through DataFrames for transformation before NetworkX

conversion, or algorithm outputs can be written back to the graph.

```
import polars as pl

# Run NetworkX algorithm
pagerank_scores = nx.pagerank(G)

# Transform to Polars DataFrame
df = pl.DataFrame({
    "name": list(pagerank_scores.keys()),
    "pagerank": list(pagerank_scores.values())
})

# Write results back to KuzuDB
conn.execute("ALTER TABLE Person ADD IF NOT EXISTS pagerank DOUBLE DEFAULT
0.0;")
conn.execute("""
    LOAD FROM df
    MERGE (p:Person {name: name})
    SET p.pagerank = pagerank;
""")
```

The `LOAD FROM` syntax directly scans DataFrames and PyArrow tables without requiring file export, providing efficient round-trip workflows between graph storage and Python analytics.

Configuration for optimal performance

KuzuDB launches as an embedded database requiring no server process. The key initialization parameters control memory and concurrency:

```
db = kuzu.Database(
    database_path="./my_database",
    buffer_pool_size=4_294_967_296,  # 4GB explicit buffer pool
    max_num_threads=8,               # Limit thread parallelism
    compression=True,                # Enable storage compression
    lazy_init=False,                 # Immediate initialization
    read_only=False                  # Read-write access
)

conn = kuzu.Connection(db, num_threads=4)
```

Buffer pool defaults to **~80% of system memory**—explicitly setting this prevents memory pressure on systems running other workloads. Runtime configuration via Cypher CALL statements allows dynamic tuning:

```
CALL THREADS=6;                      -- Thread limit for connection
CALL TIMEOUT=30000;                  -- 30-second query timeout
CALL spill_to_disk=true;             -- Enable disk spilling
CALL progress_bar=true;              -- Monitor long queries
```

Disk spilling handles larger-than-memory operations

KuzuDB automatically spills intermediate results to disk when memory pressure approaches buffer pool limits. This enables processing graphs that exceed available RAM—benchmarks show **17 billion edges loaded in ~70 minutes** with only 102GB memory available.

Disk spilling creates temporary `.tmp` files in the database directory, automatically cleaned after query completion. The feature is disabled for in-memory databases and read-only connections. For large relationship table ingestion, this capability is essential:

Buffer Memory	Load Time (17B edges, 32 threads)
420GB	1 hour
205GB	1 hour 8 minutes
102GB	1 hour 10 minutes

Embedded versus server deployment patterns

KuzuDB operates as an **embedded database by default**—the library runs in-process with no network overhead, similar to SQLite or DuckDB. This architecture provides the fastest performance but limits to a single read-write process per database directory.

For **multi-client access**, the official API server (Docker image `kuzudb/api-server`) exposes REST endpoints:

```
docker run -p 8000:8000 \
    -v /path/to/database:/database \
    -e KUZU_FILE=mydb.kuzu \
    -e MODE=READ_WRITE \
    -e KUZU_BUFFER_POOL_SIZE=2147483648 \
    --rm kuzudb/api-server:latest
```

Docker deployment also supports the Explorer UI (`kuzudb/explorer`) for visual graph exploration. Kubernetes deployments require custom manifests—no official Helm chart exists—with persistent volume claims for database storage.

The **concurrency model** requires understanding: create one `Database` object per process, spawn multiple `Connection` objects for concurrent queries, but only one write transaction executes at a time. Multiple processes can open the same database in read-only mode.

Memory-efficient patterns for large graph extraction

Converting large graphs to NetworkX requires careful memory management since NetworkX stores everything in Python dictionaries. Three strategies optimize this workflow:

Batched extraction processes subgraphs incrementally:

```
def extract_batched(conn, batch_size=100000):
    offset = 0
    while True:
        result = conn.execute(f"""
            MATCH (a)-[r]->(b)
            RETURN * SKIP {offset} LIMIT {batch_size}
        """)
        subgraph = result.get_as_networkx()
        if subgraph.number_of_edges() == 0:
            break
        yield subgraph
        offset += batch_size
```

Native algorithm extension avoids NetworkX overhead entirely—KuzuDB's `algo` extension implements PageRank, betweenness centrality, and community detection in

C++, running significantly faster than Python equivalents. Use NetworkX only for algorithms unavailable natively.

COPY TO for bulk export extracts data to Parquet for external processing:

```
COPY (MATCH (n:Person)-[r:KNOWS]->(m:Person) RETURN n.id, m.id, r.weight)
TO 'edges.parquet';
```

Ryugraph continues KuzuDB development

Following KuzuDB's archival in October 2025, **Ryugraph** (maintained by Predictable Labs) emerged as the primary active fork. It maintains API compatibility while continuing bug fixes and development under MIT license. Current release v25.9.2 (December 2025) works as a drop-in replacement.

Migration requires minimal changes—primarily updating imports and running the Ryugraph extension server for additional functionality:

```
docker run -d -p 8080:80 ghcr.io/predictable-labs/extension-repo:latest
```

```
-- Install algo extension from extension server
INSTALL algo FROM 'http://localhost:8080/';
-- Load the extension (required for each connection)
LOAD algo;
```

Note: Unlike KuzuDB v0.11.3 which has the algo extension pre-installed and pre-loaded, Ryugraph requires explicit installation from an extension server and loading per connection. The wrapper configures this via the `RYUGRAPH_EXTENSION_SERVER_URL` environment variable.

Other notable forks include **Ladybug** (community-driven "DuckDB for graphs") and **Bighorn** (Kineviz fork with GraphXR integration). The v0.11.3 release bundles common extensions (algo, fts, json, vector), ensuring existing deployments continue functioning without external dependencies.

Complete integration workflow example

This example demonstrates the full pipeline from database setup through NetworkX analysis and result persistence:

```
import kuzu
import networkx as nx
import polars as pl

# Initialize with explicit memory configuration
db = kuzu.Database("./graph_analytics", buffer_pool_size=2_147_483_648)
conn = kuzu.Connection(db)

# Create schema
conn.execute("""
    CREATE NODE TABLE IF NOT EXISTS Person(id STRING PRIMARY KEY, name STRING,
age INT64);
    CREATE REL TABLE IF NOT EXISTS Knows(FROM Person TO Person, weight DOUBLE);
""")

# Bulk load from Parquet (fastest method)
conn.execute("COPY Person FROM 'persons.parquet';")
conn.execute("COPY Knows FROM 'relationships.parquet';")

# Extract subgraph for analysis
result = conn.execute("""
    MATCH (p1:Person)-[k:Knows]->(p2:Person)
    WHERE k.weight > 0.5
    RETURN *;
""")

# Convert to NetworkX
G = result.get_as_networkx(directed=True)
print(f"Extracted {G.number_of_nodes()} nodes, {G.number_of_edges()} edges")

# Run algorithms
pagerank = nx.pagerank(G, weight='weight')
communities =
list(nx.community.greedy_modularity_communities(G.to_undirected()))

# Prepare results as DataFrame
df = pl.DataFrame({
    "id": [n.split("_")[1] for n in pagerank.keys()],
    "pagerank": list(pagerank.values())
})

# Write back to graph
conn.execute("ALTER TABLE Person ADD IF NOT EXISTS pagerank DOUBLE DEFAULT
0.0;")
conn.execute("""
    LOAD FROM df
```

```
    MERGE (p:Person {id: id})
    SET p.pagerank = pagerank;
""")

# Verify
top_nodes = conn.execute("""
    MATCH (p:Person)
    RETURN p.id, p.name, p.pagerank
    ORDER BY p.pagerank DESC LIMIT 10;
""").get_as_pl()

conn.close()
db.close()
```

Graph OLAP Platform Integration

For implementation details of how the Graph OLAP Platform uses KuzuDB/Ryugraph with NetworkX, see:

- ryugraph-wrapper.design.md - FastAPI wrapper implementation
- system.architecture.design.md - Pod architecture and data flows

Conclusion

KuzuDB (and Ryugraph) provides **production-ready NetworkX integration** through native conversion methods that preserve all graph properties. The embedded architecture delivers exceptional performance for analytical workloads—**COPY FROM ingests data ~18x faster than Neo4j**, while disk spilling enables processing of billion-edge graphs on commodity hardware. The DataFrame integration creates flexible ETL pipelines where Polars or Pandas can transform data before or after graph analysis.

Key architectural decisions: use **batch operations via COPY FROM** rather than individual inserts, configure **buffer pool explicitly** for predictable memory usage, leverage the **native algo extension** before falling back to NetworkX, and consider the **API server pattern** for multi-client deployments. With Ryugraph maintaining active development, the platform remains a viable choice for graph analytics workflows requiring Python ecosystem integration.